

Cryptanalysis of TFHE-friendly Cipher FRAST

Antoine Bak^{1,2}, Shibam Ghosh¹, Fukang Liu³, Willi Meier⁴, Jianqiang Ni⁵
and Léo Perrin¹

¹ Inria, Paris, France

{antoine.bak,shibam.ghosh,leo.perrin}@inria.fr

² Direction Générale de l’Armement (DGA), Paris, France

³ Institute of Science Tokyo, Tokyo, Japan

liu.f.ad@m.titech.ac.jp

⁴ University of Applied Sciences and Arts Northwestern Switzerland, Windisch, Switzerland

willimeier48@gmail.com

⁵ East China Normal University, Shanghai, China

jianqiangni0213@163.com

Abstract. FRAST is a TFHE-friendly stream cipher that was published at FSE 2025. The cipher is defined over $\mathbb{Z}_{16}$, and makes extensive use of negacyclic S-boxes over $\mathbb{Z}_{16}$ as they are less costly in TFHE. Like many FHE-friendly ciphers, FRAST randomizes some of its components to increase its security against statistical attacks. In the case of FRAST, some S-boxes are randomized using an XOF that takes a nonce as input. In this work, we point out a strong structural property of the full FRAST permutation, which leads to a much simpler alternative representation of the primitive. We study the consequences of this representation and find a weak key space of non-negligible size (i.e., much larger than 2^{128}) on which every ciphertext leaks one bit of plaintext. This corresponds to a distinguishing attack on the full FRAST in the weak-key setting. In particular, we emphasize that, apart from the structural property, the usage of negacyclic S-boxes further leads to a much larger weak-key space for our attack. Finally, we provide a general framework to mount a linear attack on FRAST in the average key setting. We briefly describe our approach in the end of the paper, and observe that standard assumptions expected to work in the context of linear cryptanalysis do not hold in the case of FRAST: our experiment indicate that a linear attack in the average key setting does not work as expected.

Keywords: HE-friendly cipher · stream cipher · FRAST · weak-key attack

1 Introduction

Fully homomorphic encryption (FHE) allows to perform computations (addition and multiplication) over encrypted messages, which makes it quite appealing in privacy-preserving applications. Although the poor performance of FHE schemes is still the bottleneck towards wide deployments in real-world applications, it has been subsequently improved since the first proposal by Gentry [Gen09]. Specifically, computations over encrypted messages with existing FHE schemes (e.g., BFV [FV12, Bra12], BGV [BGV14], CKKS [CKKS17], FHEW [DM15], and TFHE [CGGI20]) are still time-consuming, and the ciphertext expansion is also a problem since these FHE schemes are lattice-based.

To address the issue of ciphertext expansion, the so-called *hybrid homomorphic encryption (HHE)* or *transcipherring* framework was proposed [NLV11]. In HHE, the client side performs relatively easy computations while the server performs intensive computations. The communication cost (i.e., the ciphertext expansion) between the client and server is reduced by using a symmetric-key primitive rather than a lattice-based encryption

algorithm to encrypt plain messages. Specifically, on the client side, a symmetric-key primitive is used to encrypt the plain messages, and a FHE scheme is used to encrypt the secret key once. Then, the corresponding ciphertexts as well as the encrypted secret key are sent to the server. After receiving these, the server first performs homomorphic decryption of the ciphertexts with the encrypted secret key. Then, it performs the required homomorphic evaluation functions over the homomorphically decrypted results. Hence, as the length of plain messages increases, there is almost no ciphertext expansion since the length of the plain messages and the encrypted data sent to the server are almost the same.

Although the issue of ciphertext expansion can be addressed with HHE, performing homomorphic decryption of the received ciphertexts is necessary before starting the homomorphic evaluations. Hence, the homomorphic decryption has to be as efficient as possible in order to improve the overall performance. This leads to the emergence of the so-called FHE-friendly symmetric-key encryption algorithms, and many have been proposed in recent years. Examples include block ciphers such as LowMC [ARS⁺15], Chaghri [AMT22], and stream ciphers such as Kreyvium [CCF⁺16], FLIP [MJSC16], Rasta [DEG⁺18] and its variants Fasta [CIR22], Dasta [HL20], Pasta [DGH⁺23] and Pasta_{v2} [GLR⁺24], HERA [CHK⁺21], Rubato [HKL⁺22], Elisabeth-4 [CHMS22], FRAST [CCH⁺24] and Transistor [BBB⁺25].

In particular, many such designs use randomized components, i.e., either randomized affine layers, or randomized key schedules, or randomized S-boxes. Although these FHE-friendly ciphers do significantly outperform conventional symmetric-key primitives (e.g., AES) in FHE protocols, some do not stand the test of time or have potential weaknesses violating the security claims. A key line of cryptanalysis in this domain is algebraic attacks, which exploit the low algebraic complexity of the cipher. Notable examples include attacks on Rasta and Dasta [LSMI21, LSW⁺22], LowMC [LSMI22], Elisabeth-4 [GBJR23] and Chaghri [LAW⁺23], all of which leverage the low algebraic degree of these primitives or the sparsity of their polynomial representation. Further algebraic attacks have been mounted on Rubato [GAH⁺23] and HERA [LKSM24]. Additionally, structural weaknesses have been found in the filter function of the stream cipher FLIP [DLR16],

In this work, we continue the line of research to study the security of FHE-friendly ciphers. In particular, we focus on the TFHE-friendly stream cipher FRAST recently proposed at ToSC 2024 [CCH⁺24]. This is the first FHE-friendly symmetric-key primitive to use randomized S-boxes in the literature. Apart from this feature, it also has a quite large number of rounds (i.e., 40 rounds) compared with other such primitives, and it is defined over the integer ring $\mathbb{Z}_{16}$ rather than a finite field. Indeed, Elisabeth-4 is the first FHE-friendly stream cipher over $\mathbb{Z}_{16}$ tailored to TFHE. However, it was soon broken in [GBJR23] by exploiting the fact that the least significant bits of x, y, z are linear over $\mathbb{F}_2$ for the modular addition $z = x + y \pmod{2^i}$ where i is a positive integer.

1.1 Notations

In this paper, $\boxplus$ and $\boxminus$ represent addition and subtraction modulo 16, respectively. We sometimes also use $+$, and when necessary, the modulo will be explicitly stated. Additionally, we use $a \% b$ to denote $a \pmod{b}$. Throughout this paper, vectors and matrices are written in **bold**. For any vector $\mathbf{v} \in \mathbb{Z}_{16}^n$, the i -th entry of $\mathbf{v}$ is denoted by v_i , i.e., $\mathbf{v} = (v_1, \dots, v_n)$. We also denote $(e^1, \dots, e^{32})$ the canonical basis of $\mathbb{Z}_{16}^{32}$, where $e_j^i = 1$ if $i = j$, and 0 otherwise.

1.2 Description of FRAST

Before proceeding, we briefly review the inner workings of FRAST, as introduced in [CCH⁺24]. FRAST is a stream cipher constructed using a custom block cipher operating in a variant of the counter mode. The cipher takes as input a 256-bit master key $\mathbf{k} \in \mathbb{Z}_{16}^{64}$ and is

initialized with a 64-bit nonce $\text{NC} \in \{0, 1\}^{64}$ and a 64-bit counter $\text{CTR} \in \{0, 1\}^{64}$. The counter is incremented with each invocation of the cipher, while the nonce remains fixed for the duration of its use. The combined value $\text{NC} \parallel \text{CTR} \in \{0, 1\}^{128}$ is used to derive most of the cipher's S-boxes.

The internal block cipher consists of 40 rounds, where the r -th round is denoted by $\mathcal{R}[\mathbf{k}, \text{NC} \parallel \text{CTR}, r]$ for $1 \leq r \leq 40$, i.e.,

$$\text{FRAST}[\mathbf{k}, \text{NC} \parallel \text{CTR}] = \mathcal{R}[\mathbf{k}, \text{NC} \parallel \text{CTR}, 40] \circ \mathcal{R}[\mathbf{k}, \text{NC} \parallel \text{CTR}, 39] \circ \dots \circ \mathcal{R}[\mathbf{k}, \text{NC} \parallel \text{CTR}, 1].$$

A 128-bit keystream block is generated by encrypting a fixed input string defined as $\text{IC} = (0, 1, \dots, 15, 0, 1, \dots, 15) \in \mathbb{Z}_{16}^{32}$, i.e., the keystream is given by $z = \text{FRAST}[\mathbf{k}, \text{NC} \parallel \text{CTR}](\text{IC})$.

Round Functions of FRAST. A schematic illustration of the r -th round function $\mathcal{R}[\mathbf{k}, \text{NC} \parallel \text{CTR}, r]$ is shown in Figure 1. Let the input and output of the r -th round be $(y_1^{(r-1)}, \dots, y_{32}^{(r-1)})$ and $(y_1^{(r)}, \dots, y_{32}^{(r)})$, respectively. The round function $\mathcal{R}[\mathbf{k}, \text{NC} \parallel \text{CTR}, r]$ is defined as follows:

$$\begin{aligned} y_i^{(r)} &= y_i^{(r-1)} \boxplus S_{\text{erf}}^{(r)}(y_1^{(r-1)} \boxplus \text{rk}_j^{(r)}) \text{ for } j = 2, 3, \dots, 32, \\ y_1^{(r)} &= y_1^{(r-1)} \boxplus S_{\text{crf}}^{(r)}(\text{rk}_1^{(r)} \boxplus y_2^{(r)} \boxplus y_3^{(r)} \boxplus \dots \boxplus y_{32}^{(r)}). \end{aligned}$$

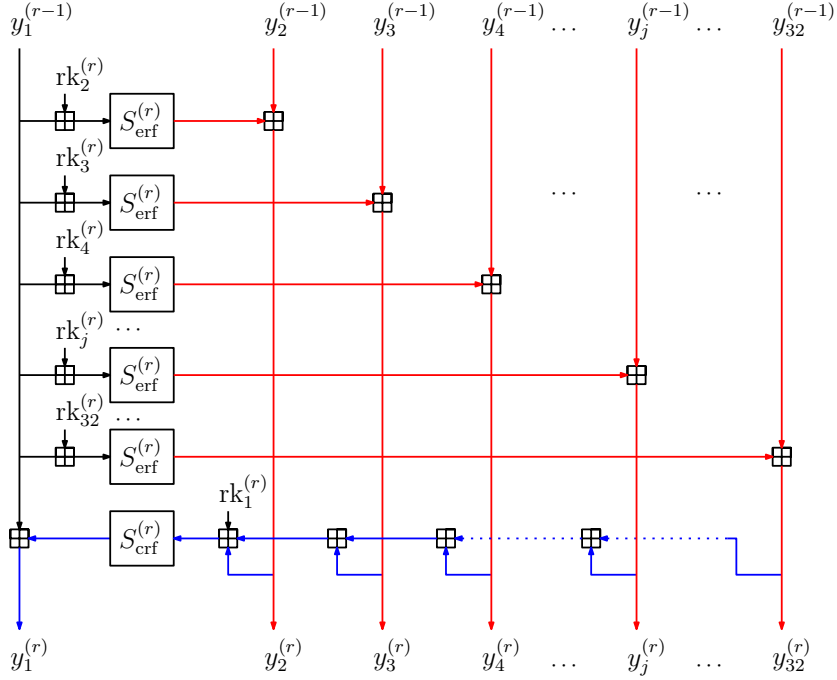


Figure 1: The round function of FRAST

S-boxes of FRAST. Crucially, the 4-bit S-boxes $S_{\text{erf}}^{(r)}$ and $S_{\text{crf}}^{(r)}$ are not fixed for 32 out of 40 rounds. Specifically, for rounds where $r \not\equiv 0 \pmod{5}$, they are generated randomly using an eXtensible Output Function (XOF) seeded with the value $\text{NC} \parallel \text{CTR}$. Since CTR increments on each call to FRAST, a unique set of S-boxes is used in each call. For rounds where $r \equiv 0 \pmod{5}$, the S-boxes are fixed and identical across those rounds. They are denoted by S_{fixed} , as defined in Table 1. Rounds that use S-boxes derived from $\text{NC} \parallel \text{CTR}$

are referred to as *random rounds*, while those using S_{fixed} are referred to as *fixed rounds*. In total, there are 32 random rounds and 8 fixed rounds.

Table 1: The fixed S-box S_{fixed} used in the fixed rounds of FRAST

y	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15
$S(y)$	0	3	5	8	6	9	12	7	13	10	14	4	1	15	11	2

Although $S_{\text{erf}}^{(r)}$ and $S_{\text{crf}}^{(r)}$ are randomly generated for each random round, only $4 \times 8 = 32$ random bits are required to instantiate each S-box. This is enabled by the property of *negacyclicity*, designed to optimize the cipher’s implementation in TFHE. Negacyclicity enforces that $S(x \boxplus 8) = \boxminus S(x)$ for all $x \in \mathbb{Z}_{16}$, which implies that only values for $x \in \{0, \dots, 7\}$ need to be explicitly sampled. We emphasize that there is no rejection when sampling these S-boxes, i.e., there are no constraints on the 32 random bits to instantiate each random S-box.

Key Schedule of FRAST. Each round key $\mathbf{rk}^{(r)} = (\text{rk}_1^{(r)}, \text{rk}_2^{(r)}, \dots, \text{rk}_{32}^{(r)}) \in \mathbb{Z}_{16}^{32}$ is a linear function of the 256-bit master key $\mathbf{k}$. Specifically, the round keys $\mathbf{rk}^{(2i-1)}$ and $\mathbf{rk}^{(2i)}$ for $i = 1, 2, \dots, 20$ are defined by $\mathbf{rk}^{(2i-1)} \parallel \mathbf{rk}^{(2i)} = \mathbf{M}^i \cdot \mathbf{k}$, where $\mathbf{M} \in \mathbb{Z}_{16}^{64 \times 64}$ is a fixed invertible matrix. For the detailed specification of $\mathbf{M}$, and for a full description of the initial security analysis of this primitive, we refer to [CCH⁺24].

1.3 Our Contributions

In this paper, we present a third-party cryptanalysis of FRAST. Our main contribution is the discovery of a strong distinguishing property in the inner block cipher of FRAST—on which all our other results are based. Indeed, it turns out that any input difference taken in a vector space of co-dimension 2 (meaning, in a space of dimension 30) will go through an arbitrary number of rounds with probability 1.

Needless to say, this property has strong consequences in terms of security. However, the mode in which the inner block cipher is used to construct a stream cipher successfully prevents some trivial attacks by injecting the counter alongside the nonce rather than through the plaintext. As a consequence, exploiting the differential patterns we identified turned out to require sophisticated techniques.

We first demonstrate that these differential patterns give rise to a large number of alternative representations of (restrictions of) the round function. These structural redundancies can be exploited in a meaningful way. Indeed, using these alternative forms, we identify extensive families of weak keys. Specifically, out of all possible 256-bit master keys, at least 2^{135+w} keys can be exploited using an online complexity of $2^{31.9}$ and an offline complexity of $2^{7(w+1)}$.

Then, we investigate the consequences of these alternative representations in terms of linear cryptanalysis. However, the linear behaviour of FRAST turns out to be poorly explained by state-of-the-art techniques. In the hopes to foster further research in this direction, we present an attempt at linear cryptanalysis, and describe the mismatch between the correlations expected based on standard arguments, and the ones we actually observed in practice.

Outline of this paper. The alternative representation of FRAST is detailed in Section 2. Then, we describe the full-round attack in the weak-key setting in Section 3. Our attempt at linear cryptanalysis in the average key setting can be found in Section 4. Finally, this paper is concluded in Section 5

2 Alternative Representations of FRAST

It turns out that the round function of FRAST admits many functionally equivalent representations, i.e., there exist multiple distinct circuit structures, in the style of Figure 1, that compute the same round function. These are consequences of a particularly strong differential property that holds with probability 1, which we detail in Section 2.1. Then, we present different alternative representations of FRAST, and in fact of the restriction of FRAST to some specific subspaces, in Section 2.2. Finally, we deduce some probability 1 linear approximations in a weak-key setting in Section 2.3.

2.1 A Powerful Differential Distinguisher

FRAST is built as a generalized Feistel network operating on 32 branches. However, the specific variant used in FRAST exhibits an unusually strong property. Using the notation from Figure 1, consider two plaintexts: $(y_1, \dots, y_{32})$ and $(y_1, y_2 \boxplus \delta, y_3 \boxminus \delta, y_4, \dots, y_{32})$, meaning a pair corresponding to the input differential $(0, \delta, \boxminus \delta, 0, \dots, 0)$. This differential pattern passes through the first part of the round function, namely the layer involving $S_{\text{erf}}^{(r)}$, since the difference does not affect any of the S-box inputs. However, it also remains unchanged in the second part. Indeed, the input to $S_{\text{erf}}^{(r)}$ is the same for both plaintexts because the summed difference is equal to 0: $\delta - \delta = 0$. As a consequence, for any δ , we have an iterated probability 1 differential.

This pattern can be greatly generalized. The choice to place the non-zero differences in the second and third branches is arbitrary, and many other such configurations are possible. In fact, the following result holds.

Theorem 1 (Probability 1 Iterated Differentials). *Let $\Delta = (0, \Delta_2, \Delta_3, \dots, \Delta_{32})$ be a vector of $\mathbb{Z}_{16}^{32}$ such that $\sum_{i=2}^{32} \Delta_i = 0$. Then, for any input $x \in \mathbb{Z}_{16}^{32}$, key $\mathbf{k}$, nonce NC , and round number r , the following equation holds:*

$$\mathcal{R}[\mathbf{k}, NC || CTR, r](x + \Delta) = \mathcal{R}[\mathbf{k}, NC, r](x) + \Delta. \quad (1)$$

In other words, the round function preserves the difference Δ with probability 1, as long as two linear constraints are satisfied: $\Delta_1 = 0$ and $\sum_{i=2}^{32} \Delta_i = 0$. The set of such valid differences forms a subspace of dimension 30 over $\mathbb{Z}_{16}$, meaning Theorem 1 applies to 2^{120} distinct input differences out of all 2^{128} possible.

2.2 Describing Alternative Representations

Representing a Restriction. To get a better understanding of Theorem 1, we describe in this section an alternative representation of the round function of FRAST. For this purpose, we divide one round of FRAST into two steps: the *expanding* step and the *compressing* step. Then, we apply each step on a state $(x_1, \dots, x_{32})$, and study how it modifies the specific pair $(x_1, \sum_{k=2}^{32} x_k)$.

- **Expanding step.** This step is defined as

$$(x_1, \dots, x_{32}) \mapsto (x_1, x_2 \boxplus S_{\text{erf}}(x_1 \boxplus \text{rk}_2), \dots, x_{32} \boxplus S_{\text{erf}}(x_1 \boxplus \text{rk}_{32})).$$

After this step, the pair $(x_1, \sum_{k=2}^{32} x_k)$ becomes

$$\left(x_1, \sum_{i=2}^{32} (x_i \boxplus S_{\text{erf}}(x_1 \boxplus \text{rk}_i)) \right) = \left(x_1, \sum_{i=2}^{32} x_i \boxplus \sum_{j=2}^{32} S_{\text{erf}}(x_1 \boxplus \text{rk}_j) \right).$$

- **Compressing step.** This step is defined as

$$(x_1, \dots, x_{32}) \mapsto \left(x_1 \boxplus S_{\text{crf}} \left(\sum_{k=2}^{32} x_k \boxplus \text{rk}_1 \right), x_2, \dots, x_{32} \right).$$

After this step, the pair $(x_1, \sum_{k=2}^{32} x_k)$ becomes

$$\left(x_1 \boxplus S_{\text{crf}} \left(\sum_{k=2}^{32} x_k \boxplus \text{rk}_1 \right), \sum_{k=2}^{32} x_k \right).$$

Now, if we consider $F_1: x \mapsto \sum_{j=2}^{32} S_{\text{erf}}(x \boxplus \text{rk}_j)$ and $F_2: x \mapsto S_{\text{crf}}(x \boxplus \text{rk}_1)$, then it is clear from the above discussion that one round $\mathcal{R}[\mathbf{k}, \text{NC} || \text{CTR}, r]$ of FRAST acts on $(x_1, \sum_{k=2}^{32} x_k)$ as two rounds of a two-branch Feistel with round functions F_1 and F_2 . If we denote this two branch Feistel as $g[\mathbf{k}, \text{NC} || \text{CTR}, r]$, and if we let $\mathcal{L}(x_1, \dots, x_{32}) = (x_1, \sum_{k=2}^{32} x_k)$, then

$$\mathcal{L} \circ \mathcal{R}[\mathbf{k}, \text{NC} || \text{CTR}, r](x_1, \dots, x_{32}) = g[\mathbf{k}, \text{NC} || \text{CTR}, r] \circ \mathcal{L}(x_1, \dots, x_{32}).$$

This two-branch Feistel structure is represented as the two leftmost branches in Figure 2. Recall that $(e^1, \dots, e^{32})$ is the canonical basis of $\mathbb{Z}_{16}^{32}$. Using it, we reframe this finding as follows: the subspace with basis $(e^1, \sum_{k>1} e^k)$ is stable under the round function of FRAST, and said restriction can be seen as a two-branched Feistel network.

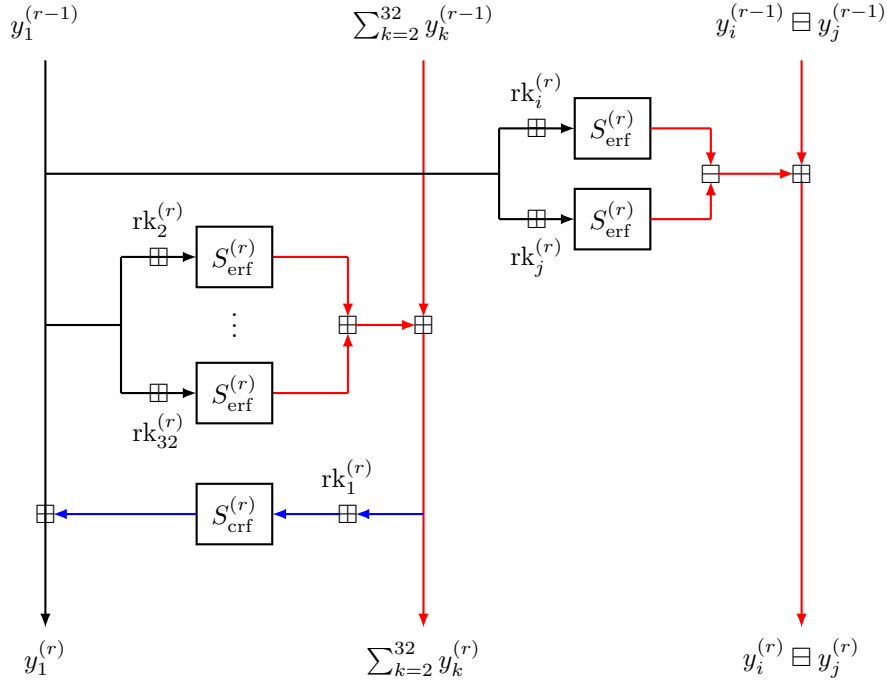


Figure 2: An alternative representation of a restriction of the FRAST round function.

Considering other restrictions. Another important observation is that after one round of FRAST, for $2 \leq i \leq 32$, x_i is mapped to $x_i \boxplus S_{\text{erf}}^{(r)}(x_1 \boxplus \text{rk}_i^{(r)})$, which only depends on the coordinates x_1 and x_i of the input. We will show that this observation allows the description of highly structured alternative representations of the whole FRAST, and also of its restrictions to other spaces.

In its original definition, the round functions are defined using the canonical basis $(e^1, \dots, e^{32})$ of $\mathbb{Z}_{16}^{32}$. Let us write the equations defining FRAST over other bases. For all $2 \leq i < j \leq 32$, we introduce a subspace of $\mathbb{Z}_{16}^{32}$ defined by the basis

$$\mathcal{B}_{(i,j)} = \left(e^1, \sum_{k=2}^{32} e^k, e^i - e^j \right).$$

Over the corresponding subspace of dimension 3, the input differences Δ that satisfy the conditions of Theorem 1 take the form $(0, 0, \delta)$, meaning that the non-zero coordinates lie only in the last coordinate. We then define an alternative representation of the round function as follows:

$$\mathcal{R}^{\text{alt}}[\mathbf{k}, \text{NC} || \text{CTR}, r](x_1, x_2, x_3) = (g[\mathbf{k}, \text{NC} || \text{CTR}, r](x_1, x_2), f_{(i,j)}[\mathbf{k}, \text{NC} || \text{CTR}, r](x_1, x_3)). \quad (2)$$

Here, the function $g[\mathbf{k}, \text{NC} || \text{CTR}, r]$ corresponds to the two-branch Feistel structure we defined earlier, i.e. it can be decomposed into two parts as follows

$$g[\mathbf{k}, \text{NC} || \text{CTR}, r](x_1, x_2) = g_2[\mathbf{k}, \text{NC} || \text{CTR}, r] \circ g_1[\mathbf{k}, \text{NC} || \text{CTR}, r](x_1, x_2),$$

where g_1 and g_2 are such that

$$\begin{aligned} g_1[\mathbf{k}, \text{NC} || \text{CTR}, r](x_1, x_2) &= \left(x_1, x_2 \boxplus \sum_{i=2}^{32} S_{\text{erf}}^{(r)}(x_1 \boxplus \text{rk}_i^{(r)}) \right) \\ g_2[\mathbf{k}, \text{NC} || \text{CTR}, r](x_1, x_2) &= \left(x_1 \boxplus S_{\text{erf}}^{(r)}(x_2 \boxplus \text{rk}_1^{(r)}), x_2 \right). \end{aligned}$$

The last output coordinate (the rightmost one in Figure 2) is then defined as:

$$f_{(i,j)}[\mathbf{k}, \text{NC} || \text{CTR}, r](x_1, x_3) = x_3 \boxplus S_{\text{erf}}^{(r)}(x_1 \boxplus \text{rk}_i^{(r)}) \boxminus S_{\text{erf}}^{(r)}(x_1 \boxplus \text{rk}_j^{(r)}). \quad (3)$$

This structure mirrors the choice of the basis $\mathcal{B}_{(i,j)}$, where the last vector is expressed as a difference between two standard basis vectors. Thus, the relationship between the original round function $\mathcal{R}$ and its alternative representation $\mathcal{R}^{\text{alt}}$ is captured by the fact that they commute with a specific linear mapping:

$$\mathcal{L} \circ \mathcal{R}[\mathbf{k}, \text{NC}, r](x_1, \dots, x_{32}) = \mathcal{R}^{\text{alt}}[\mathbf{k}, \text{NC}, r] \circ \mathcal{L}(x_1, \dots, x_{32}), \quad (4)$$

where $\mathcal{L}$ is the linear transformation mapping the canonical representation of vectors to the alternative representation. As a result, the subset $(z_1, \sum_{k=2}^{32} z_k, z_i \boxminus z_j)$ of the output of the full FRAST cipher can be equivalently expressed using:

$$\mathcal{L} \circ \text{FRAST}[\mathbf{k}, \text{NC} || \text{CTR}] = \mathcal{R}^{\text{alt}}[\mathbf{k}, \text{NC} || \text{CTR}, 40] \circ \dots \circ \mathcal{R}^{\text{alt}}[\mathbf{k}, \text{NC} || \text{CTR}, 1] \circ \mathcal{L}.$$

Remark 1. If the round function had included a final swap between branches — such as the branch rotation commonly used in many generalized Feistel networks — then the iterative structure underlying these alternative representations would no longer hold. It is therefore reasonable to attribute this structural vulnerability to the absence of such a swap. However, whether introducing a swap would be sufficient to eliminate the issue entirely remains an open question.

2.3 Probability 1 Linear Approximations

As a consequence of the observation above, the alternative representation can be used to construct a linear approximation of the round function. Consider the structure depicted in

Figure 2 with respect to $\mathcal{B}_{(i,j)}$. Let $\mathbf{x}^{(r-1)}, \mathbf{x}^{(r)} \in \mathbb{Z}_{16}^3$ denote the input and output vectors of $\mathcal{R}^{\text{alt}}$ at round r . From the definition of $f_{(i,j)}$ in Equation (3), we have:

$$\begin{aligned} x_3^{(r)} &= f_{(i,j)}[\mathbf{k}, \text{NC} || \text{CTR}, r](x_1^{(r-1)}, x_3^{(r-1)}) \\ &= x_3^{(r-1)} \boxplus \left(S_{\text{erf}}^{(r)}(x_1^{(r-1)} \boxplus \text{rk}_i^{(r)}) \boxminus S_{\text{erf}}^{(r)}(x_1^{(r-1)} \boxplus \text{rk}_j^{(r)}) \right). \end{aligned}$$

Now suppose the round key satisfies $\text{rk}_i^{(r)} = \text{rk}_j^{(r)}$. In that case, the two S-box inputs are identical, and the subtraction cancels out, leading to: $x_3^{(r)} = x_3^{(r-1)}$. If this condition holds for every round $r \in [1, 40]$, we obtain the following invariant over 40 rounds of the alternative representation: $x_3^{(40)} = x_3^{(0)}$. Translating this result back to the canonical basis (i.e., undoing the change of basis), if $\mathbf{x}^{(40)} = \mathcal{L}(\mathbf{y}^{(40)})$ and $\mathbf{x}^{(0)} = \mathcal{L}(\mathbf{y}^{(0)})$, then we find the following relation in the input and output values of 40-round FRAST:

$$y_i^{(40)} \boxminus y_j^{(40)} = \text{IC}_i \boxminus \text{IC}_j,$$

where $(\text{IC}_i, \text{IC}_j)$ are the corresponding coordinates of the initial input, which is public. In particular, we have the following linear relation:

$$y_i^{(40)} + y_j^{(40)} = \text{IC}_i + \text{IC}_j \pmod{2},$$

which we will use in the attack. This relation holds for any choice of $\text{NC} || \text{CTR}$, provided the condition $\text{rk}_i^{(r)} = \text{rk}_j^{(r)}$ is satisfied for all rounds. For the remainder of the paper, given an alternative representation (i.e., a fixed basis $\mathcal{B}_{(i,j)}$), we refer to any master key that satisfies the corresponding conditions as a *weak-key* with respect to that representation.

Once again, we emphasize that this iterative relation critically depends on the absence of branch swaps. If a swap (such as a branch rotation) were introduced at the end of each round — as is common in many Feistel-based constructions — this structural invariance would no longer hold.

3 Refined Weak-key Attack on Full FRAST

In this section, we elaborate on the weak-key attack with the goal of expanding the weak-key space such that the probability of encountering a weak key exceeds 2^{-128} . This threshold is significant because the designers of FRAST only claim 128-bit security, despite using a 256-bit master key. We believe that if the number of weak keys exceeds 2^{128} , the attack becomes more practically relevant and exposes a meaningful structural vulnerability in the cipher.

The above analysis given in Section 2.2 indicates that whatever S_{erf} is, it is always possible to add 4 bit conditions on round keys at each round to construct a deterministic full-round distinguisher. However, this attack will cause $4 \times 40 = 160$ bit conditions. Let us first assume these conditions are independent. As there are $\binom{31}{2}$ possible choices of $\mathcal{B}$, we can have $\binom{31}{2}$ alternative representations. Hence, there are $\binom{31}{2}$ different sets of weak keys, and each set is of size of about $2^{256-160} = 2^{96}$. If the intersection of each set is small enough compared with 2^{96} , the weak key space size is estimated as

$$\binom{31}{2} \cdot 2^{256-160} = \binom{31}{2} \cdot 2^{96} < 2^{128}.$$

How to further enlarge the weak key space so that a weak key is used with probability higher than 2^{-128} ? In the following, we will describe three methods to solve this problem.

3.1 Exploiting the Negacyclic Property of Random S-boxes

As already stated, there are 32 rounds of FRAST where random S-boxes are used. In particular, each random S-box is a negacyclic function over $\mathbb{Z}_{16}$. Hence, Lemma 1 will be useful to reduce the number of conditions on round keys for the random rounds.

Lemma 1. *For a negacyclic function $S_{\text{erf}}^{(r)}$ over $\mathbb{Z}_{16}$ which satisfies*

$$\forall x \in \mathbb{Z}_{16} : S_{\text{erf}}^{(r)}(x) = \boxminus S_{\text{erf}}^{(r)}(x \boxplus 8) ,$$

and for any $\delta \in \{0, 8\}$, the following holds:

$$\forall x \in \mathbb{Z}_{16} : S_{\text{erf}}^{(r)}(x \boxplus \delta) - S_{\text{erf}}^{(r)}(x) = 0 \pmod{2} . \quad (5)$$

Proof. The proof is rather simple:

- If $\delta = 0$, then it is clear that $S_{\text{erf}}^{(r)}(x \boxplus \delta) \boxminus S_{\text{erf}}^{(r)}(x) = 0$. In particular, the equality also holds modulo 2.
- If $\delta = 8$, then by negacyclicity, we have that $S_{\text{erf}}^{(r)}(x \boxplus \delta) \boxminus S_{\text{erf}}^{(r)}(x) = \boxminus 2 \cdot S_{\text{erf}}^{(r)}(x) = 0 \pmod{2}$.

□

Specifically, any random round $\mathcal{R}^{\text{alt}}[\mathbf{k}, \text{NC} || \text{CTR}, r]$ and for any $\mathcal{B}_{(i,j)}$, we have

$$\begin{aligned} x_3^{(r)} &= f_{(i,j)}[\mathbf{k}, \text{NC} || \text{CTR}, r](x_1^{(r-1)}, x_3^{(r-1)}) \\ &= x_3^{(r-1)} \boxplus \left(S_{\text{erf}}^{(r)}(x_1^{(r-1)} \boxplus \text{rk}_i^{(r)}) \boxminus S_{\text{erf}}^{(r)}(x_1^{(r-1)} \boxplus \text{rk}_j^{(r)}) \right) \\ &= x_3^{(r-1)} + \left(S_{\text{erf}}^{(r)}(x_1^{(r-1)} \boxplus \text{rk}_i^{(r)}) + S_{\text{erf}}^{(r)}(x_1^{(r-1)} \boxplus \text{rk}_j^{(r)}) \right) \pmod{2} . \end{aligned} \quad (6)$$

Thus, if $\text{rk}_i^{(r)} \boxminus \text{rk}_j^{(r)} \in \{0, 8\}$ then (according to Lemma 1) we have the following important consequence: $x_3^{(r)} = x_3^{(r-1)} \pmod{2}$. Therefore, instead of imposing the 4 bit conditions $\text{rk}_i^{(r)} = \text{rk}_j^{(r)} \pmod{16}$ for each random round, we only need to impose 3 bit conditions by exploiting the negacyclic property of the random S-boxes. However, the condition for the 8 fixed rounds remain the same. In this way, there are still $\binom{31}{2}$ different sets of weak keys, and each set is of size

$$2^{256-32 \cdot 3-8 \cdot 4} = 2^{128} .$$

If the intersection of these sets is small enough, the weak-key space size is estimated as

$$\binom{31}{2} \cdot 2^{128} > 2^{128} .$$

In this sense, the claim for 128-bit security does not seem to hold for FRAST.

3.2 Generating Weak Random S-boxes with Chosen Nonce

As is clear in our weak-key attack, for any r -th round $\mathcal{R}^{\text{alt}}$ and for any three-branch structure $\mathcal{B}_{(i,j)}$, Equation (6) holds. It implies that $x_3^{(r)} = x_3^{(r-1)} \pmod{2}$ provided that $\text{rk}_i^{(r)} = \text{rk}_j^{(r)} \pmod{16}$. If $S_{\text{erf}}^{(r)}(x)$ is a negacyclic function, the condition can be improved to $\text{rk}_i^{(r)} = \text{rk}_j^{(r)} \pmod{8}$. Both cases have been discussed above. Are there other methods that can further relax the condition? In the following, we describe the second method to enlarge the weak-key space.

During each random round, each negacyclic function $S_{\text{erf}}^{(r)}(x)$ over $\mathbb{Z}_{16}$ is randomly generated by an XOF seeded with NC||CTR. Moreover, due to the negacyclic property, only the values of $S_{\text{erf}}^{(r)}(0), S_{\text{erf}}^{(r)}(1), \dots, S_{\text{erf}}^{(r)}(7)$ are randomly sampled, and $S_{\text{erf}}^{(r)}(8), S_{\text{erf}}^{(r)}(9), \dots, S_{\text{erf}}^{(r)}(15)$ are accordingly determined. Hence, 32 random bits are required to specify each random 4-bit S-box $S_{\text{erf}}^{(r)}(x)$. With this mind, we can choose NC||CTR such that *weak* random S-boxes are generated, for a definition of “weak” that will be clarified further below. The negacyclic property imposes that

$$\forall x \in \mathbb{Z}_{16} : S_{\text{erf}}^{(r)}(x) = \boxminus S_{\text{erf}}^{(r)}(x \boxplus 8) \implies S_{\text{erf}}^{(r)}(x) = S_{\text{erf}}^{(r)}(x \boxplus 8) \pmod{2}.$$

Hence, if

$$S_{\text{erf}}^{(r)}(0) = S_{\text{erf}}^{(r)}(1) = \dots = S_{\text{erf}}^{(r)}(7) \pmod{2}, \quad (7)$$

then it will hold that

$$S_{\text{erf}}^{(r)}(0) = S_{\text{erf}}^{(r)}(1) = \dots = S_{\text{erf}}^{(r)}(7) = S_{\text{erf}}^{(r)}(8) = S_{\text{erf}}^{(r)}(9) = \dots = S_{\text{erf}}^{(r)}(15) \pmod{2}.$$

An S-box for which Equation (7) holds is called *weak* in what follows. This property occurs with probability $2^{-8+1} = 2^{-7}$, meaning each random S-box is a weak S-box in our weak-key attack with probability 2^{-7} . If a weak random S-box $S_{\text{erf}}^{(r)}(x)$ is used at the r -th round then we have

$$\forall x_1^{(r-1)}, \text{rk}_i^{(r)}, \text{rk}_j^{(r)} \in \mathbb{Z}_{16} : S_{\text{erf}}^{(r)}(x_1^{(r-1)} \boxplus \text{rk}_i^{(r)}) + S_{\text{erf}}^{(r)}(x_1^{(r-1)} \boxplus \text{rk}_j^{(r)}) = 0 \pmod{2}.$$

Hence, there is no need to add conditions on the round key at this round, and the following deterministic iterative relation is still preserved:

$$x_3^{(r)} = x_3^{(r-1)} \pmod{2}.$$

Counting the number of weak keys and possible combinations. Let w be the number of weak S-boxes $S_{\text{erf}}^{(r)}(x)$ at $w \leq 32$ random rounds. Then, there are $\binom{32}{w}$ possible combinations to choose rounds instantiated with weak $S_{\text{erf}}^{(r)}(x)$. Each combination will cause a full-round weak-key attack. In each combination, there are still $\binom{31}{2}$ different sets of weak keys, and each set is of size

$$2^{256 - (32-w) \cdot 3 \cdot 8 \cdot 4} = 2^{128+3w}.$$

This indicates that even if there is only one random S-box satisfying Equation (7), i.e., $w = 1$, then weak key occurs with probability higher than 2^{-128} .

Complexity to generate each combination of w weak S-boxes. Generating weak S-boxes can be done offline, as it is only related to NC and the XOF. For each combination, there are w S-boxes required to be weak and their positions are fixed. As a weak S-box occurs with probability 2^{-7} , the offline complexity to generate w weak S-boxes for w pre-determined random rounds is 2^{7w} . In the following, we study the time complexity of the weak-key attack if the random rounds with weak random S-boxes are pre-determined.

Overall complexity. The offline complexity of our attack is 2^{7w} . The online complexity is $\binom{31}{2}$, i.e., we have $\binom{31}{2}$ possible three-branch structure $\mathcal{B}_{(i,j)}$. To filter the false positive cases, we need to repeat the online phase a few times. Assume the online phase is repeated ϵ times, which implies that we also need to collect ϵ different NC that can lead to w weak S-boxes at the same w random rounds. Hence, the offline complexity becomes $\epsilon \cdot 2^{7w}$ and

the online complexity becomes $\epsilon \cdot \binom{31}{2}$. Then, if a weak key is used, then there will be one $\mathcal{B}_{(i,j)}$ such that the following relation always holds:

$$x_3^{(40)} = x_3^{(0)} \pmod{2} \implies y_i^{(40)} + y_j^{(40)} = y_i^{(0)} + y_j^{(0)} = IC_i + IC_j \pmod{2},$$

for ϵ tests. If the key is *not* weak, then this holds for ϵ tests with a probability of about $2^{-\epsilon}$. For simplicity, we set $\epsilon = 128 = 2^7$.

For each set of weak keys, if we aim to balance the offline cost and the probability that a weak key appears, we can choose w such that

$$2^{7+7w} \approx 2^{256} \cdot \frac{1}{2^{128+3w}} = 2^{128-3w}.$$

This gives the optimal value $w = 12$. In this case, for each set of weak keys, the offline complexity is 2^{96} and a weak key occurs with probability 2^{-92} . The online complexity is $2^7 \cdot \binom{31}{2} = 2^{15.9}$.

If we aim to maximize the probability that a weak key occurs, we can choose $w = 17$. In this case, the offline complexity is 2^{126} , and a weak key occurs in each set with probability 2^{-77} . Note that if the intersection of the weak-key sets is small enough compared with the size of each weak-key set, the probability that a weak key occurs can be estimated as $\binom{31}{2} \cdot 2^{-77} \approx 2^{-68.1}$.

3.3 Add Key-recovery Rounds

The weak key space can be further increased by a factor of 2^7 at the cost of increasing the complexity of the online phase. In the previous analysis, we have stated that there are several sets of weak keys and the sets are divided according to the choice of w rounds instantiated with weak S-boxes. For each set of weak keys, the offline complexity of the attack is $\epsilon \cdot 2^{7w}$ and the online complexity is $\epsilon \cdot \binom{31}{2}$. In this basic attack, to detect whether a weak key is used, we simply test whether there exists a three-branch structure with $\mathcal{B}_{(i,j)}$ such that $x_3^{(40)} = x_3^{(0)} \pmod{2}$ always holds for ϵ tests. This test can be improved by involving a key-recovery strategy, which is our third method. In other words, in our third method to improve the weak-key space, we can not only improve the weak-key space, but also recover partial key bits.

Specifically, in the key-recovery attack on full FRAST, we assume that $\text{rk}_i^{(1)} \neq \text{rk}_j^{(1)} \pmod{8}$ at the first round and $\text{rk}_i^{(40)} \neq \text{rk}_j^{(40)}$ at the last round¹. Note that the first round is instantiated with random S-boxes, while the last round is instantiated with the fixed S-box. Assume that we have found ϵ different NC that can cause w weak S-boxes at the same pre-determined w random rounds where the first round is not included. This is done offline and the complexity is still $\epsilon \cdot 2^{7w}$.

At the online phase, for each NC||CTR, we get output of the form $\mathbf{y}^{(40)}$. At first we transform it into alternative basis $\mathbf{x}^{(40)} = \mathcal{L}(\mathbf{y}^{(40)})$. Thus, we have the required outputs:

$$(x_1^{(40)}, x_2^{(40)}, x_3^{(40)}).$$

Then, guess $(\text{rk}_i^{(1)}, \text{rk}_j^{(1)}, \text{rk}_i^{(40)}, \text{rk}_j^{(40)}, \text{rk}_1^{(40)})$ where

$$\text{rk}_i^{(1)} \neq \text{rk}_j^{(1)} \pmod{8}, \text{rk}_i^{(40)} \neq \text{rk}_j^{(40)}.$$

¹If assuming $\text{rk}_i^{(1)} = \text{rk}_j^{(1)} \pmod{8}$ and $\text{rk}_i^{(40)} \neq \text{rk}_j^{(40)}$, this will be a similar key-recovery attack where the last round is used for key recovery. If assuming $\text{rk}_i^{(1)} \neq \text{rk}_j^{(1)} \pmod{8}$ and $\text{rk}_i^{(40)} = \text{rk}_j^{(40)}$, only the first round is used for key recovery. If both hold, no need to add key-recovery rounds.

Therefore, there are about 2^{20} candidates to guess (in fact, slightly less than 2^{20} candidates). For each guess, we want to compute $x_i^{(1)}$ and $x_i^{(39)}$ so that we can check if $x_i^{(1)} = x_i^{(39)} \pmod{2}$. We compute $x_i^{(1)}$ and $x_i^{(39)}$ as follows:

$$x_3^{(1)} = x_3^{(0)} \boxplus \left(S_{\text{erf}}^{(1)}(x_1^{(0)} \boxplus \text{rk}_i^{(1)}) \boxminus S_{\text{erf}}^{(1)}(x_1^{(0)} \boxplus \text{rk}_j^{(1)}) \right) \quad (8)$$

$$\begin{aligned} x_1^{(39)} &= x_1^{(40)} \boxminus \left(S_{\text{fixed}}(x_2^{(40)} \boxplus \text{rk}_1^{(40)}) \right) \\ x_3^{(39)} &= x_3^{(40)} \boxminus \left(S_{\text{erf}}^{(1)}(x_1^{(39)} \boxplus \text{rk}_i^{(40)}) \boxminus S_{\text{erf}}^{(40)}(x_1^{(39)} \boxplus \text{rk}_j^{(40)}) \right). \end{aligned} \quad (9)$$

If weak round keys are used in the middle $38 - w$ rounds, for the correct guess of the 20 key bits, the following relation

$$x_3^{(1)} = x_3^{(39)} \pmod{2} \quad (10)$$

always holds for any NC||CTR. However, due to the special form of Equation (10), there are more than one correct keys left. We call *equivalently correct keys*. For other key guesses, Equation (10) will hold with probability 2^{-1} , and hence it holds for ϵ tests with probability $2^{-\epsilon}$. In our setting, $\epsilon = 128$.

Equivalently correct keys. From Equation (8), it is clear that guessing $(\text{rk}_i^{(1)}, \text{rk}_j^{(1)}) = (v_0, v_1) \in \mathbb{Z}_{16}^2$ and $(\text{rk}_i^{(1)}, \text{rk}_j^{(1)}) = (v_1, v_0) \in \mathbb{Z}_{16}^2$ are equivalent and will result in the same value for $x_3^{(1)} \pmod{2}$. This similarly applies to the guess of $(\text{rk}_2^{(40)}, \text{rk}_3^{(40)})$ for computing $x_3^{(39)}$. In other words, if the guess $(\text{rk}_i^{(1)}, \text{rk}_j^{(1)}, \text{rk}_2^{(40)}, \text{rk}_3^{(40)}) = (v_0, v_1, v_2, v_3)$ always leads to (10) for ϵ chosen nonce, it is redundant to test the following 3 candidates

$$(v_0, v_1, v_3, v_2), (v_1, v_0, v_2, v_3), (v_1, v_0, v_3, v_2)$$

since the 4 candidates cannot be distinguished by (10). This reduces the key space to guess by a factor of 4, but also increases the candidates left for the correct key by a factor of 4.

Moreover, as $S_{\text{erf}}^{(1)}(x)$ is a negacyclic function satisfying $\forall x \in \mathbb{Z}_{16} : S_{\text{erf}}^{(1)}(x) = \boxminus S_{\text{erf}}^{(1)}(x \boxplus 8)$, we have $\forall x \in \mathbb{Z}_{16} : S_{\text{erf}}^{(1)}(x) = S_{\text{erf}}^{(1)}(x \boxplus 8) \pmod{2}$. Therefore, we only need to guess $(\text{rk}_i^{(1)}, \text{rk}_j^{(1)}) = (v_0, v_1)$ satisfying $0 \leq v_0, v_1 < 8$. This is because after guessing such a (v_0, v_1) , we also get

$$\begin{aligned} &\left(S_{\text{erf}}^{(1)}(x_1^{(0)} \boxplus \text{rk}_i^{(1)}) + S_{\text{erf}}^{(1)}(x_1^{(0)} \boxplus \text{rk}_j^{(1)}) \right) \\ &= \left(S_{\text{erf}}^{(1)}(x_1^{(0)} \boxplus \text{rk}_i^{(1)} \boxplus 8) + S_{\text{erf}}^{(1)}(x_1^{(0)} \boxplus \text{rk}_j^{(1)} \boxplus 8) \right) \pmod{2}. \end{aligned}$$

In other words, the following 4 candidates for $(\text{rk}_i^{(1)}, \text{rk}_j^{(1)})$ cannot be distinguished by Equation 10:

$$(v_0, v_1), (v_0 \boxplus 8, v_1), (v_0, v_1 \boxplus 8), (v_0 \boxplus 8, v_1 \boxplus 8).$$

As a consequence, this further reduces the key space to guess by a factor of 4, but also further increases the number of candidates for the correct key by a factor of 4. In summary, due to the equivalently correct key for Equation (10), we do not need to exhaust about 2^{20} candidates. Instead, only less than 2^{16} candidates need to be tested.

Complexity evaluation. The complexity of the offline phase is $\epsilon \cdot 2^{7w} = 2^{7w+7}$. The online complexity is $2^{16} \cdot \epsilon \cdot \binom{31}{2} = 2^{23} \cdot \binom{31}{2}$. There are still $\binom{31}{2}$ sets of weak keys, and each set is of size

$$2^{256 - (31-w) \cdot 3 - 7 \cdot 4} = 2^{135+3w}.$$

This increases the number of weak keys by a factor of 2^7 compared with the second method under the same value of w . The offline complexity remains the same as for the second method, but the online complexity of the third method increases by a factor of 2^{16} .

3.4 The Number of Weak Keys

In the above analysis, we assume that each condition

$$\text{rk}_i^{(r)} = \text{rk}_j^{(r)} \pmod{16} \text{ or } \text{rk}_i^{(r)} = \text{rk}_j^{(r)} \pmod{8}$$

reduces the weak key space by a factor of 2^4 and 2^3 , respectively. Is this a reasonable estimate? In this section, we will address this issue.

First, according to the key schedule of FRAST, each round key nibble $\text{rk}_i^{(r)}$ is represented by a linear polynomial over $\mathbb{Z}_{16}$ in the master key $\mathbf{k} = (k_1, k_2, \dots, k_{64}) \in \mathbb{Z}_{16}^{64}$. Moreover, $\text{rk}_i^{(r)} = \text{rk}_j^{(r)} \pmod{8}$ is equivalent to

$$\text{rk}_i^{(r)} \boxminus \text{rk}_j^{(r)} = 0 \text{ or } \text{rk}_i^{(r)} \boxminus \text{rk}_j^{(r)} = 8.$$

Hence, all conditions can be written as a set of linear equations in $(k_1, k_2, \dots, k_{64})$ over $\mathbb{Z}_{16}$.

For the three-branch structure with $\mathcal{B}_{(i,j)}$, we consider the case where there are 40 conditions specified in Equation 11, i.e., note that this corresponds to increasing weak keys with the negacyclic property of the random S-boxes.

$$\text{rk}_i^{(r)} \boxminus \text{rk}_j^{(r)} = \delta_r, \quad \delta_r = \begin{cases} 0, & \text{if } r \equiv 0 \pmod{5}, \\ 0 \text{ or } 8, & \text{otherwise.} \end{cases} \quad (11)$$

The linear equation system over $\mathbb{Z}_{16}$ corresponding to these 40 conditions is denoted by

$$\mathbf{M}_{i,j} \cdot \mathbf{k} = \mathbf{b} \pmod{16}, \quad (12)$$

where $\mathbf{M}_{i,j} \in \mathbb{Z}_{16}^{40 \times 64}$ is the coefficient matrix, and $\mathbf{b} \in \mathbb{Z}_{16}^{40}$ is parameterized by $\boldsymbol{\delta} = (\delta_1, \dots, \delta_{40}) \in \mathbb{Z}_{16}^{40}$. Note that $\mathbf{b}$ can take 2^{32} possible values since there are 32 random rounds.

Studying the weak key set for the three-branch structure with $\mathcal{B}_{(i,j)}$ is equivalent to studying the solution space of the linear equation system Equation 12. If the equation system is defined over a finite field, computing the rank of $\mathbf{M}_{i,j}$ is enough. However, Equation 12 is over the integer ring $\mathbb{Z}_{16}$. Even if $\mathbf{M}_{i,j}$ is of full rank, i.e., all rows are linearly independent over $\mathbb{Z}_{16}$, it does not necessarily mean that there are $16^{64-40} = 2^{96}$ solutions for any $\mathbf{b}$.

The number of solutions to Equation 12. For each $\mathbf{M}_{i,j}$, we use Gaussian elimination to find 40 columns that can form an invertible square matrix over $\mathbb{Z}_{16}$, and denote the matrix formed by these 40 columns of $\mathbf{M}_{i,j}$ by $\mathbf{M}_{i,j}^{\text{left}} \in \mathbb{Z}_{16}^{40 \times 40}$. The matrix formed by the remaining $64 - 40 = 24$ columns of $\mathbf{M}_{i,j}$ is denoted by $\mathbf{M}_{i,j}^{\text{right}} \in \mathbb{Z}_{16}^{40 \times 24}$. Specifically, after applying Gaussian elimination to $\mathbf{M}_{i,j}^{\text{left}}$ to obtain its reduced row echelon form, it can become the identity matrix. It is found that for each possible (i, j) , it is always possible to find such an invertible square matrix $\mathbf{M}_{i,j}^{\text{left}}$. This implies that the number of solutions to Equation 12 is exactly $2^{96+32} = 2^{128}$. Specifically, Equation 12 can always be rewritten as

$$\mathbf{M}_{i,j}^{\text{left}} \cdot \mathbf{k}_L = \mathbf{b}' - \mathbf{M}_{i,j}^{\text{right}} \cdot \mathbf{k}_R \pmod{16}, \quad (13)$$

where $\mathbf{k}_L \in \mathbb{Z}_{16}^{40}$, $\mathbf{k}_R \in \mathbb{Z}_{16}^{24}$. Since the matrix

$$\left[\mathbf{M}_{i,j}^{\text{left}} \mid \mathbf{M}_{i,j}^{\text{right}} \right]$$

is obtained by permuting the columns of $\mathbf{M}_{i,j}$, $\begin{pmatrix} \mathbf{k}_L \\ \mathbf{k}_R \end{pmatrix}$ and $\mathbf{b}'$ are obtained by permuting the rows of $\mathbf{k}$ and $\mathbf{b}$, respectively. Note that $\mathbf{b}'$ can take 2^{32} possible values, while $\mathbf{k}_R$ can

take $16^{24} = 2^{96}$ possible values. As $\mathbf{M}_{i,j}^{\text{left}}$ is invertible over $\mathbb{Z}_{16}$, there are in total 2^{128} solutions to $\mathbf{k}$.

The above analysis also indicates that the number of solutions is 2^{128+3w} if removing w equations from Equation 13 with the second method, and that the number of solutions is 2^{135+3w} if removing $w + 2$ equations from Equation 13 with the third method. Hence, the claim for the size of each set of weak keys in the above analysis is correct.

Intersection between different sets of weak keys. If the intersection between different sets of weak keys is small enough compared with each set, after applying the third method, it is reasonable to claim that the total number of weak keys is about

$$\binom{31}{2} \cdot 2^{135+3w} = 2^{143.9+3w}.$$

Indeed, we can interpret the intersection between two different sets of weak keys from the perspective of the solutions to a larger linear equation system. For two sets corresponding to the coefficient matrices $\mathbf{M}_{i,j}$ and $\mathbf{M}_{i^*,j^*}$ where $(i,j) \neq (i^*,j^*)$, the intersection between the two sets corresponds to the solutions to the following linear equation system:

$$\begin{pmatrix} \mathbf{M}_{i,j} \\ \mathbf{M}_{i^*,j^*} \end{pmatrix} \cdot \mathbf{k} = \begin{pmatrix} \mathbf{b} \\ \mathbf{b}^* \end{pmatrix} \pmod{16},$$

where $\begin{pmatrix} \mathbf{b} \\ \mathbf{b}^* \end{pmatrix}$ can take 2^{64} possible values. However, as can be observed, compared with the case where one set of weak keys is studied, the number of equations to consider here is doubled. Hence, even if there are solutions to this linear equation system, the number is too small compared with 2^{128} . To partially verify this, for all $\binom{31}{2} = 10780$ possible combinations, we have checked that the number of solutions is smaller than 100 when the right-hand side is set to zero. Hence, we believe that it is reasonable to estimate the number of weak keys as $2^{143.9+3w}$. Or we can simply use a conservative estimate, that is, we could consider only the number of weak keys in each set, which is a lower bound on the number of weak keys, i.e., the lower bound is 2^{135+3w} .

4 An Attempt at Linear Cryptanalysis

In this section, we present a general framework for the linear cryptanalysis of FRAST, which we also use to reinterpret the weak-key attack from the previous section. In particular, our objective is to perform a linear cryptanalysis in the average key setting, as detailed in Section 4.1. However, when conducting experiments, we observe that some common assumptions in statistical cryptanalysis do not hold in our attack. We discuss those in Section 4.2. Although our linear cryptanalysis fails in the average key setting, we chose to report our findings as we find this failure to be interesting in itself as it provides new information about the limits of commonly made assumptions.

Using the alternative representation from Section 2, we suggest the following game to model an attack against FRAST reduced to m rounds.

1. Randomly sample a vector $(t_1, \dots, t_m)$ from $\mathbb{Z}_{16}^m$.
2. Collect T_{precomp} tuples of random S-boxes. The adversary can choose N_{data} of them.
3. For each chosen tuple of S-boxes, a vector $(x_1, \dots, x_m)$, is sampled at random. The adversary then gets tuples of the form

$$\left((S_{\text{erf}}^{(r)})_{1 \leq r \leq m}, \sum_{r=1}^m S_{\text{erf}}^{(r)}(x_r \boxplus t_r) \boxminus S_{\text{erf}}^{(r)}(x_r) \right). \quad (14)$$

The objective of the game for the adversary is then to obtain some information on the key, or at least to distinguish it from the same vector where the sum on the right is replaced with a uniformly random value over $\mathbb{Z}_{16}$.

For simplicity, in the following, we simply write $S_{\text{erf}}^{(r)}$ as $S^{(r)}$. Note that if i is a multiple of 5, $S^{(r)} = S_{\text{fixed}}$. Otherwise, it is a random negacyclic S-box. Also, since there are N_{data} such vectors shown in Equation 14, to distinguish the S-boxes $(S_{\text{erf}}^{(r)})_{1 \leq r \leq m}$ used in the u -th vector, they are denoted by

$$\mathbf{S}_u = (S_u^{(1)}, \dots, S_u^{(m)})$$

where at the u -th vector, $S_u^{(r)} = S_{\text{fixed}}$ if $r \% 5 = 0$ and $S_u^{(r)}$ is a random negacyclic S-box if $r \% 5 \neq 0$.

Connection with the weak-key attack. The previous weak-key attack corresponds to multiplying the sum on the right by 8 such that it has correlation 1 over some special weak-key space, e.g.,

$$t_r = 0 \pmod{8} \text{ for } r \% 5 \neq 0, \text{ and } t_r = 0 \text{ for } r \% 5 = 0.$$

Specifically, in Section 3, we have shown that, under some conditions on the key, the least significant bit of some coordinates will be constant through all keystream elements. Over $\mathbb{Z}_{16}$, the least significant bit of x is 0 (resp. 1) if and only if $8x$ is 0 (resp. 8). Thus, our weak-key attack can be reinterpreted as a linear attack with mask 8 that gives a linear approximation of correlation 1 over some weak-key space, and under some right guess of key material/choices of nonce.

4.1 Linear Cryptanalysis in the Average Key Setting

A natural question is how to extend this attack to the average key setting? In its most general form, we can divide the attack into the following steps:

1. Take T_{precomp} nonce and compute the corresponding random S-boxes. Keep N_{data} of them according to some criteria.
2. Query these N_{data} nonce to get $(\mathbf{y}_i^{(m)}, \mathbf{y}_j^{(m)})$ where

$$\mathbf{y}_i^{(m)} = (y_{i,1}^{(m)}, \dots, y_{i,N_{\text{data}}}^{(m)}), \mathbf{y}_j^{(m)} = (y_{j,1}^{(m)}, \dots, y_{j,N_{\text{data}}}^{(m)}).$$

For the u -th nonce, $(y_{i,u}^{(m)}, y_{j,u}^{(m)})$ denote the output keystream words at the i -th and j -th branches. In addition, $y_{i,u}^{(1)}, y_{j,u}^{(1)}, y_{i,u}^{(m-1)}, y_{j,u}^{(m-1)}$ denote the outputs at the i -th and j -th branches after 1 round and $m-1$ rounds, respectively.

3. Guess some key material in the external rounds to obtain

$$(y_{i,u}^{(1)}, y_{j,u}^{(1)}, y_{i,u}^{(m-1)}, y_{j,u}^{(m-1)}),$$

i.e., as in our key-recovery attack in the weak-key setting. In addition, guess $t_r = \text{rk}_i^{(r)} \boxminus \text{rk}_j^{(r)}$ for the internal rounds where $2 \leq r \leq m-1$.

4. For each guess, compute a “corrected” empirical correlation defined as

$$\hat{c} = \frac{1}{N_{\text{data}}} \sum_{u=1}^{N_{\text{data}}} g(\mathbf{S}_u, t_2, \dots, t_{m-1}) \cdot \zeta_{16}^{a \cdot \tilde{x}_{3,u}}.$$

where $\zeta_{16} = e^{2i\pi/16}$ is a primitive 16-th root of unity, $a \in \mathbb{Z}_{16}$ is a mask, g is a function with values in $\mathbb{C}$, and $x_{3,u}$ is the value of the difference

$$\tilde{x}_{3,u} = (y_{i,u}^{(m-1)} \boxminus y_{j,u}^{(m-1)}) \boxminus (y_{i,u}^{(1)} \boxminus y_{j,u}^{(1)})$$

after having removed the first and last round according to the key guess at the first and last rounds. If the key guesses at the first and last rounds are correct, then we have

$$\tilde{x}_{3,u} = \sum_{r=2}^{m-1} S_u^{(r)}(x_{1,u}^{(r)} \boxplus \text{rk}_i^{(r)}) \boxminus S_u^{(r)}(x_{1,u}^{(r)} \boxplus \text{rk}_j^{(r)}) .$$

5. Keep the key guesses that give correlation greater than some threshold.

Our attack. In the following, we denote the expectation of $h: X \rightarrow \mathbb{C}$ as

$$\mathbb{E}_{x \in X}(h(x)) = \frac{1}{|X|} \sum_{x \in X} h(x) ,$$

and define the correlation of $f: \mathbb{Z}_{16} \rightarrow \mathbb{Z}_{16}$ as

$$C^{f(x)} = \mathbb{E}_{y \in \mathbb{Z}_{16}} \left(\zeta_{16}^{f(y)} \right) = \frac{1}{16} \sum_{y \in \mathbb{Z}_{16}} \zeta_{16}^{f(y)} .$$

In our attack, we have selected our N_{data} nonces in order to maximize for each nonce, the *expected linear potential* (ELP) over the $m - 2$ middle rounds, i.e. the quantity

$$\prod_{r=2}^{m-1} \mathbb{E}_{t_r \in \mathbb{Z}_{16}} \left(\left| C^{a \cdot (S^{(r)}(x \boxplus t_r) \boxminus S^{(r)}(x))} \right|^2 \right) .$$

For the key recovery function g , we chose

$$g(\mathbf{S}_u, t_2, \dots, t_{m-1}) = \text{sgn} \left(\prod_{r=2}^{m-1} \overline{C^{a \cdot (S_u^{(r)}(x \boxplus t_r) \boxminus S_u^{(r)}(x))}} \right) ,$$

where $\text{sgn}(\rho e^{i\theta}) = e^{i\theta}$, $\rho \in \mathbb{R}_{>0}$ and $\text{sgn}(0) = 0$. This function has been chosen so that, under the right key guess (which we denote $\mathbf{t}$)², the expectation of the empirical correlation becomes:

$$c = \mathbb{E}_{(x_{1,u}^{(r)})_{\substack{1 \leq u \leq N_{\text{data}} \\ 2 \leq r \leq m-1}}} (\hat{c}) = \frac{1}{N_{\text{data}}} \sum_{u=1}^{N_{\text{data}}} \prod_{r=2}^{m-1} |C^{a \cdot (S_u^{(r)}(x \boxplus t_r) \boxminus S_u^{(r)}(x))}| . \quad (15)$$

Finally, we chose the threshold to be close to the average square correlation threshold we fixed during the precomputation step in order to remove a lot of wrong key candidates.

4.2 Limits of the Assumptions Made

We tried to implement our attack on a version of FRAST reduced to 5 rounds. We made the tradeoffs between the precomputation and online phase so that both take approximately the same time. By guessing the first and last round key, our attack makes statistics on the 3 internal random rounds. We made the following (usual) right and wrong key assumptions:

²It means that the guessed key material for the first/last rounds and the key difference at the middle rounds are correct.

- **Right key:** When guessing the right key, we obtain random variables with correlation c , where c is defined as in Equation (15).
- **Wrong key:** When guessing the wrong key, we obtain random variables with correlation close to 0.

Following [BSV07], we set the threshold and data complexity of the attack in order to filter out all keys except a small constant number, and so that this small set contains the right key with probability at least 90%. However, we found experimentally that our attack, and in particular the wrong key assumption, did not work as expected. In particular, while the thresholds and data complexity we set should guarantee that we only keep a small number of wrong key candidates, our experiments let a non negligible fraction of them pass the filter. We tested the following explanations for where the problem could come from.

In the precomputation step. This step achieves the correlations we expect for the S-boxes, so it is unlikely that the problem comes from this step.

In the right key assumption. The right key passed the threshold in all our experiments. This tends to show that our right key assumption is realistic.

In the independence of the round keys. This assumption (which is common in statistical attacks) seems reasonable in our case. Indeed, our precomputation step poses no conditions on the values of the random S_{crf} . Thus, the XOF randomizes the inputs given to the S_{erf} differences that appear after the random rounds. The only remaining places where round independence could not hold is after fixed round. We however do not think it is sufficient to explain our phenomena.

In the wrong key assumption. Having a lot of wrong keys passing the filter suggests that the problems come from the wrong key assumption. To simulate the correlations we get in the case of wrong key guesses, we took S-boxes generated by our precomputation step and computed for different t_1, t_2 and $2 \leq r \leq 4$ the quantity

$$\mathbb{E}_{1 \leq u \leq N_{\text{data}}} \left(g(S_u^{(r)}, t_1) \cdot C^{a \cdot (S_u^{(r)}(x \boxplus t_2) \boxminus S_u^{(r)}(x))} \right),$$

for values of $t_1 \neq t_2$. This quantity can be thought of as the contribution of round r to the bias when the right key is t_2 and we guess t_1 . The wrong key assumption implies that this quantity should be close to 0 when $t_2 \neq t_1$. However, we observed experimentally that those quantities may have nontrivial biases when making a wrong key guess. This is the most plausible explanation for our failure in performing linear cryptanalysis.

We leave as an open problem a proper explication of this phenomenon which, so far, has remained outside of our grasp.

5 Conclusion

The FRAST stream cipher has a very strong structural flaw, making it feasible to find a simpler alternative representation. This representation allowed us to find a large weak key space on the full primitive, which is made even larger by the negacyclic property of the random S-boxes.

In the average key setting, we made a first attempt at linear cryptanalysis by describing a general framework to work around the fact that new random S-boxes are generated at each call to the cipher. However, when applying it to a reduced version of the primitive, we observed that some wrong key assumptions did not hold. We think that an interesting open problem would be to have a deeper understanding of the phenomena we observed, and to mount a working linear cryptanalysis. This would provide a better understanding of the security margin of FRAST in this setting.

References

- [AMT22] Tomer Ashur, Mohammad Mahzoun, and Dilara Toprakhisar. Chaghri - A FHE-friendly Block Cipher. In *CCS*, pages 139–150. ACM, 2022.
- [ARS⁺15] Martin R. Albrecht, Christian Rechberger, Thomas Schneider, Tyge Tiessen, and Michael Zohner. Ciphers for MPC and FHE. In *EUROCRYPT (1)*, volume 9056 of *Lecture Notes in Computer Science*, pages 430–454. Springer, 2015.
- [BBB⁺25] Jules Baudrin, Sonia Belaïd, Nicolas Bon, Christina Boura, Anne Canteaut, Gaëtan Leurent, Pascal Paillier, Léo Perrin, Matthieu Rivain, Yann Rotella, and Samuel Tap. Transistor: a TFHE-friendly Stream Cipher. *IACR Cryptol. ePrint Arch.*, page 282, 2025.
- [BGV14] Zvika Brakerski, Craig Gentry, and Vinod Vaikuntanathan. (Leveled) Fully Homomorphic Encryption without Bootstrapping. *ACM Trans. Comput. Theory*, 6(3):13:1–13:36, 2014.
- [Bra12] Zvika Brakerski. Fully Homomorphic Encryption without Modulus Switching from Classical GapSVP. In *CRYPTO*, volume 7417 of *Lecture Notes in Computer Science*, pages 868–886. Springer, 2012.
- [BSV07] Thomas Baignères, Jacques Stern, and Serge Vaudenay. Linear cryptanalysis of non binary ciphers: (with an application to SAFER). In *International Workshop on Selected Areas in Cryptography*, pages 184–211. Springer, 2007.
- [CCF⁺16] Anne Canteaut, Sergiu Carpov, Caroline Fontaine, Tancrede Lepoint, María Naya-Plasencia, Pascal Paillier, and Renaud Sirdey. Stream Ciphers: A Practical Solution for Efficient Homomorphic-Ciphertext Compression. In *FSE*, volume 9783 of *Lecture Notes in Computer Science*, pages 313–333. Springer, 2016.
- [CCH⁺24] Mingyu Cho, Woohyuk Chung, Jincheol Ha, Jooyoung Lee, Eun-Gyeol Oh, and Mincheol Son. FRAST: TFHE-Friendly Cipher Based on Random S-Boxes. *IACR Trans. Symmetric Cryptol.*, 2024(3):1–43, 2024.
- [CGGI20] Iliaria Chillotti, Nicolas Gama, Mariya Georgieva, and Malika Izabachène. TFHE: Fast Fully Homomorphic Encryption Over the Torus. *J. Cryptol.*, 33(1):34–91, 2020.
- [CHK⁺21] Jihoon Cho, Jincheol Ha, Seongkwang Kim, ByeongHak Lee, Joohee Lee, Jooyoung Lee, Dukjae Moon, and Hyojin Yoon. Transciphering Framework for Approximate Homomorphic Encryption. In *ASIACRYPT (3)*, volume 13092 of *Lecture Notes in Computer Science*, pages 640–669. Springer, 2021.
- [CHMS22] Orel Cosseron, Clément Hoffmann, Pierrick Méaux, and François-Xavier Standardt. Towards Case-Optimized Hybrid Homomorphic Encryption - Featuring the Elisabeth Stream Cipher. In *ASIACRYPT (3)*, volume 13793 of *Lecture Notes in Computer Science*, pages 32–67. Springer, 2022.
- [CIR22] Carlos Cid, John Petter Indrøy, and Håvard Raddum. FASTA - A Stream Cipher for Fast FHE Evaluation. In *CT-RSA*, volume 13161 of *Lecture Notes in Computer Science*, pages 451–483. Springer, 2022.
- [CKKS17] Jung Hee Cheon, Andrey Kim, Miran Kim, and Yong Soo Song. Homomorphic Encryption for Arithmetic of Approximate Numbers. In *ASIACRYPT (1)*, volume 10624 of *Lecture Notes in Computer Science*, pages 409–437. Springer, 2017.

- [DEG⁺18] Christoph Dobraunig, Maria Eichlseder, Lorenzo Grassi, Virginie Lallemand, Gregor Leander, Eik List, Florian Mendel, and Christian Rechberger. Rasta: A Cipher with Low ANDdepth and Few ANDs per Bit. In *CRYPTO (1)*, volume 10991 of *Lecture Notes in Computer Science*, pages 662–692. Springer, 2018.
- [DGH⁺23] Christoph Dobraunig, Lorenzo Grassi, Lukas Helminger, Christian Rechberger, Markus Schafneger, and Roman Walch. Pasta: A Case for Hybrid Homomorphic Encryption. *IACR Trans. Cryptogr. Hardw. Embed. Syst.*, 2023(3):30–73, 2023.
- [DLR16] Sébastien Duval, Virginie Lallemand, and Yann Rotella. Cryptanalysis of the FLIP Family of Stream Ciphers. In *CRYPTO (1)*, volume 9814 of *Lecture Notes in Computer Science*, pages 457–475. Springer, 2016.
- [DM15] Léo Ducas and Daniele Micciancio. FHEW: Bootstrapping Homomorphic Encryption in Less Than a Second. In *EUROCRYPT (1)*, volume 9056 of *Lecture Notes in Computer Science*, pages 617–640. Springer, 2015.
- [FV12] Junfeng Fan and Frederik Vercauteren. Somewhat Practical Fully Homomorphic Encryption. *IACR Cryptol. ePrint Arch.*, page 144, 2012.
- [GAH⁺23] Lorenzo Grassi, Irati Manterola Ayala, Martha Norberg Hovd, Morten Øygarden, Håvard Raddum, and Qingju Wang. Cryptanalysis of symmetric primitives over rings and a key recovery attack on rubato. In *CRYPTO (3)*, volume 14083 of *Lecture Notes in Computer Science*, pages 305–339. Springer, 2023.
- [GBJR23] Henri Gilbert, Rachelle Heim Boissier, Jérémy Jean, and Jean-René Reinhard. Cryptanalysis of Elisabeth-4. In *ASIACRYPT (3)*, volume 14440 of *Lecture Notes in Computer Science*, pages 256–284. Springer, 2023.
- [Gen09] Craig Gentry. Fully homomorphic encryption using ideal lattices. In *STOC*, pages 169–178. ACM, 2009.
- [GLR⁺24] Lorenzo Grassi, Fukang Liu, Christian Rechberger, Fabian Schmid, Roman Walch, and Qingju Wang. Minimize the Randomness in Rasta-Like Designs: How Far Can We Go? - Application to Pasta. In *SAC (2)*, volume 15517 of *Lecture Notes in Computer Science*, pages 207–238. Springer, 2024.
- [HKL⁺22] Jincheol Ha, Seongkwang Kim, ByeongHak Lee, Jooyoung Lee, and Mincheol Son. Rubato: Noisy Ciphers for Approximate Homomorphic Encryption. In *EUROCRYPT (1)*, volume 13275 of *Lecture Notes in Computer Science*, pages 581–610. Springer, 2022.
- [HL20] Phil Hebborn and Gregor Leander. Dasta - Alternative Linear Layer for Rasta. *IACR Trans. Symmetric Cryptol.*, 2020(3):46–86, 2020.
- [LAW⁺23] Fukang Liu, Ravi Anand, Libo Wang, Willi Meier, and Takanori Isobe. Coefficient Grouping: Breaking Chaghri and More. In *EUROCRYPT (4)*, volume 14007 of *Lecture Notes in Computer Science*, pages 287–317. Springer, 2023.
- [LKSM24] Fukang Liu, Abul Kalam, Santanu Sarkar, and Willi Meier. Algebraic Attack on FHE-Friendly Cipher HERA Using Multiple Collisions. *IACR Trans. Symmetric Cryptol.*, 2024(1):214–233, 2024.
- [LSMI21] Fukang Liu, Santanu Sarkar, Willi Meier, and Takanori Isobe. Algebraic Attacks on Rasta and Dasta Using Low-Degree Equations. In *ASIACRYPT (1)*, volume 13090 of *Lecture Notes in Computer Science*, pages 214–240. Springer, 2021.

- [LSMI22] Fukang Liu, Santanu Sarkar, Willi Meier, and Takanori Isobe. The Inverse of χ and Its Applications to Rasta-Like Ciphers. *J. Cryptol.*, 35(4):28, 2022.
- [LSW⁺22] Fukang Liu, Santanu Sarkar, Gaoli Wang, Willi Meier, and Takanori Isobe. Algebraic Meet-in-the-Middle Attack on LowMC. In *ASIACRYPT (1)*, volume 13791 of *Lecture Notes in Computer Science*, pages 225–255. Springer, 2022.
- [MJSC16] Pierrick Méaux, Anthony Journault, François-Xavier Standaert, and Claude Carlet. Towards Stream Ciphers for Efficient FHE with Low-Noise Ciphertexts. In *EUROCRYPT (1)*, volume 9665 of *Lecture Notes in Computer Science*, pages 311–343. Springer, 2016.
- [NLV11] Michael Naehrig, Kristin E. Lauter, and Vinod Vaikuntanathan. Can homomorphic encryption be practical? In *CCSW*, pages 113–124. ACM, 2011.